# Soccer Twos Training

Alexia-Madalina Cirstea, Diego Yang

Check Phase 2 Progress for more information on how to train the agents and how everything was set up!

Example learning command:

mlagents-learn "C:\Users\User\Desktop\Project 2-1\ml-agents_AIML1\config\poca\SoccerTwos.yaml" --env="C:\Users\User\Desktop\Project 2-1\ml-agents_AIML1\config\UnityEnvironment.exe" --run-id=soccer-run-X --results-dir="C:\Users\User\Desktop\Project 2-1\ml-agents_AIML1\results" --num-envs=1

- Where soccer-run-X means the number of the run (to differentiate between them)

Example result command:
tensorboard --logdir="C:\Users\User\Desktop\Project 2-1\ml-agents_AIML1\results\soccer-run-X"

- Where soccer-run-X means the number of the run (to differentiate between them)

Details:

- soccer-run-4 is for POCA with the given YAML file (initial run) finished writing
- soccer-run-5 is for POCA with the given YAML file (second run, but started from 0, no continuation of the previous training) finished writing
- soccer-run-6 is for PPO with a self-made YAML file finished writing
- soccer-run-7 is for POCA with changed parameters for the YAML file //todo
- soccer-run-8 is for POCA with changed parameters for the YAML file //todo
- soccer-run-9 is for PPO with changed parameters for the YAML file //todo

# Soccer-run-4: Analysis and Statistics

## *POCA RL Algorithm*

**The YAML file:**

```yaml
behaviors:
  SoccerTwos:
    trainer_type: poca
    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: constant
    network_settings:
      normalize: false
      hidden_units: 512
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
    keep_checkpoints: 5
    max_steps: 50000000
    time_horizon: 1000
    summary_freq: 10000
    self_play:
      save_steps: 50000
      team_change: 200000
      swap_steps: 2000
      window: 10
      play_against_latest_model_ratio: 0.5
      initial_elo: 1200.0
```

## The TensorBoard Results

- TensorBoard4

## Analysis:

### 1. Cumulative Reward Distribution

- **Observed Range**: The cumulative reward distribution spans from around **-1.0** to **-0.05**, with peaks near **-0.35** and **-0.15**.
- **Interpretation**:
    - This distribution suggests that the agents are facing challenges in achieving high positive rewards consistently. The predominance of values in the slightly negative range implies that agents are managing to avoid highly negative outcomes but are not consistently winning or achieving high rewards.
    - The clustering around the range of **-0.35 to -0.15** shows that agents are generally stable but not optimized for frequent positive results, potentially due to the competitive and adversarial nature of the SoccerTwos environment.
- **Impact of POCA**: POCA's centralized critic likely assists agents in stabilizing their strategies against opposing teams, which helps in avoiding drastic losses. However, the persistent slightly negative rewards indicate that agents are still refining their competitive strategies.

### 2. Group Cumulative Reward Distribution

- **Observed Range**: The group cumulative reward histogram shows a broader spread, with significant peaks at both **negative** and **positive** values, spanning from approximately **-0.95** to **+0.95**.
- **Interpretation**:
    - This distribution suggests that, while individual agents are often experiencing slightly negative rewards, the cumulative outcomes for groups of agents (e.g., teams) vary widely. Peaks in both the positive and negative regions indicate that teams experience a mix of wins and losses, reflecting the adversarial and competitive dynamics of the environment.
    - The spread of values in both directions highlights that group performance is less consistent, with fluctuations based on the evolving strategies of each team.
- **Impact of POCA**: The centralized critic enables agents within a team to coordinate effectively, resulting in a variety of outcomes. The presence of positive group rewards suggests that, while individual agents may have slightly negative scores, the coordinated team efforts lead to a mix of winning and losing episodes.

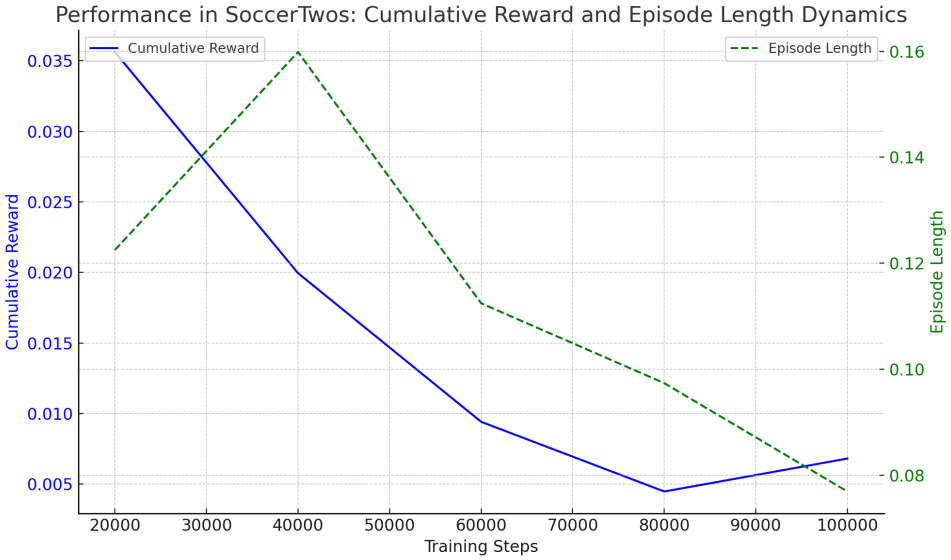
## 3. ACombined Analysis of Cumulative Reward and Group Cumulative Reward

- The discrepancy between individual cumulative rewards (predominantly negative) and group rewards (spread across positive and negative) suggests that **POCA is facilitating team-level strategy coordination**. Even if individual agents are not consistently positive, the combined efforts of the team occasionally lead to positive outcomes.
- **Learning Dynamics**: POCA's centralized critic helps agents learn not just individually but in coordination with teammates. This may lead to oscillating outcomes where teams sometimes win (positive group rewards) and sometimes lose, reflecting adaptive strategies that are still stabilizing.

## Key Insights

- **Individual Stability with Slight Negativity**: Agents achieve slightly negative rewards individually, suggesting stable but conservative strategies.
- **Group Variation**: The spread in group cumulative rewards shows that POCA facilitates team-based exploration, leading to wins in some episodes and losses in others.
- **Coordination in Multi-Agent Settings**: POCA's centralized critic likely plays a role in enabling teamwork, which is crucial in the SoccerTwos environment where teams compete directly.

In summary, POCA's design for multi-agent environments is evident in the performance metrics, with individual agents maintaining stable but slightly negative rewards while teams as a whole experience varied outcomes. This analysis suggests that while agents are not yet optimized for consistent high performance, they are effectively coordinating as teams, leading to a range of competitive outcomes.



# Soccer-run-5: Analysis and Statistics

## *POCA RL Algorithm*

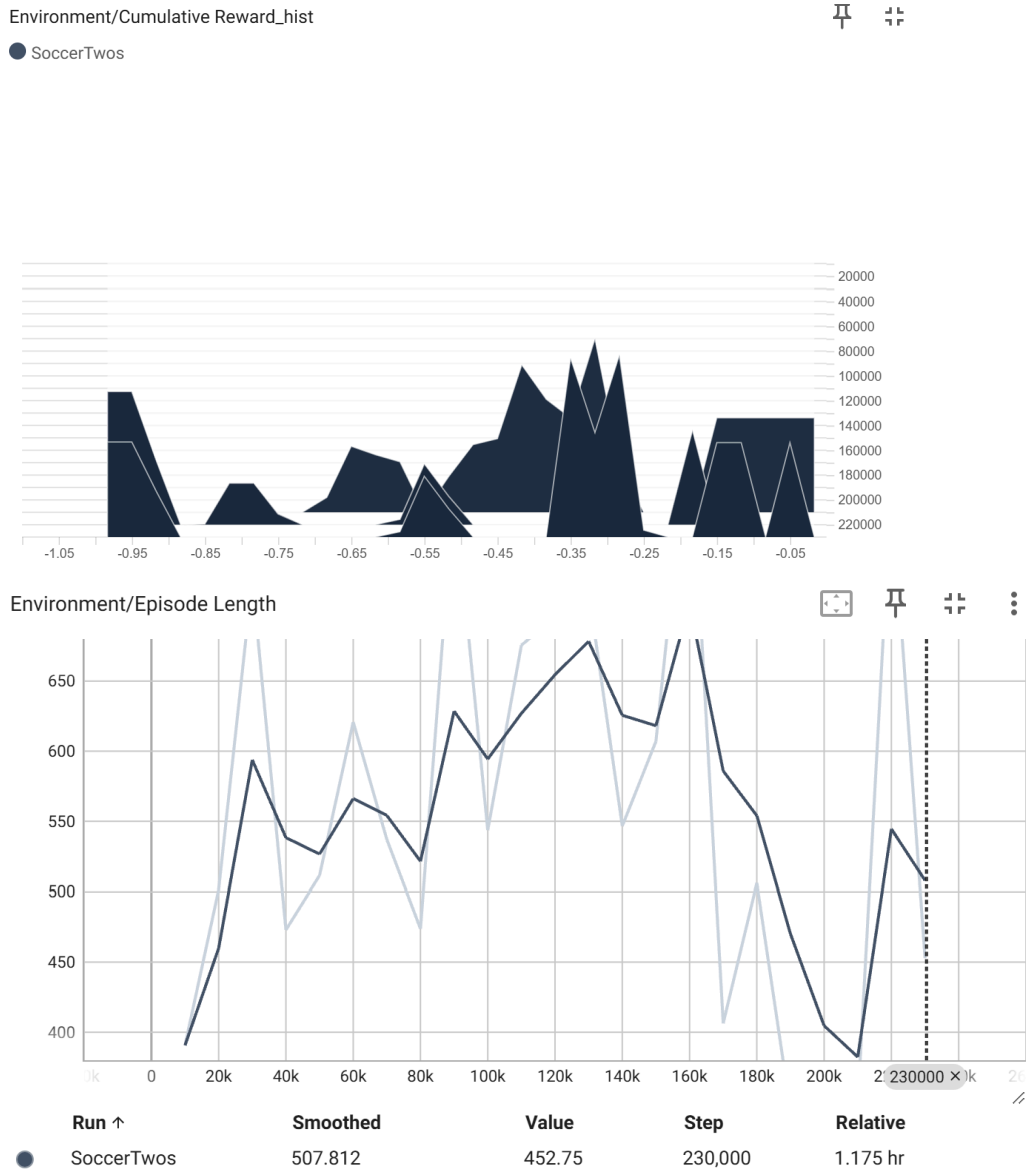
**The YAML file:**

```yaml
behaviors:
  SoccerTwos:
    trainer_type: poca
    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: constant
    network_settings:
      normalize: false
      hidden_units: 512
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
    keep_checkpoints: 5
    max_steps: 50000000
    time_horizon: 1000
    summary_freq: 10000
    self_play:
      save_steps: 50000
      team_change: 200000
      swap_steps: 2000
      window: 10
      play_against_latest_model_ratio: 0.5
```

```
      initial_elo: 1200.0
```

## The TensorBoard Results

The Tensorboard results:




Click on the following links to check the data:

- TensorBoard5

## Analysis:

**Cumulative Reward Distribution (Histogram)**

- The histogram of cumulative rewards shows a range of values primarily between **-1.0 and -0.05**, with a significant concentration around the **-0.35 to -0.15** range.
- **Interpretation**: This distribution suggests that, on average, agents are experiencing slightly negative outcomes in the SoccerTwos environment. While they aren't achieving positive rewards consistently, the rewards are clustering near zero, indicating that agents are avoiding highly detrimental actions.
- **POCA's Impact**: The centralized critic in POCA likely helps agents adapt to the multi-agent competitive dynamics, which explains why agents manage to stay within a relatively narrow reward range and avoid severe penalties. However, the negative skew indicates that agents may still be in the exploration or learning phase, trying to find effective strategies.

**2. Episode Length Over Time**

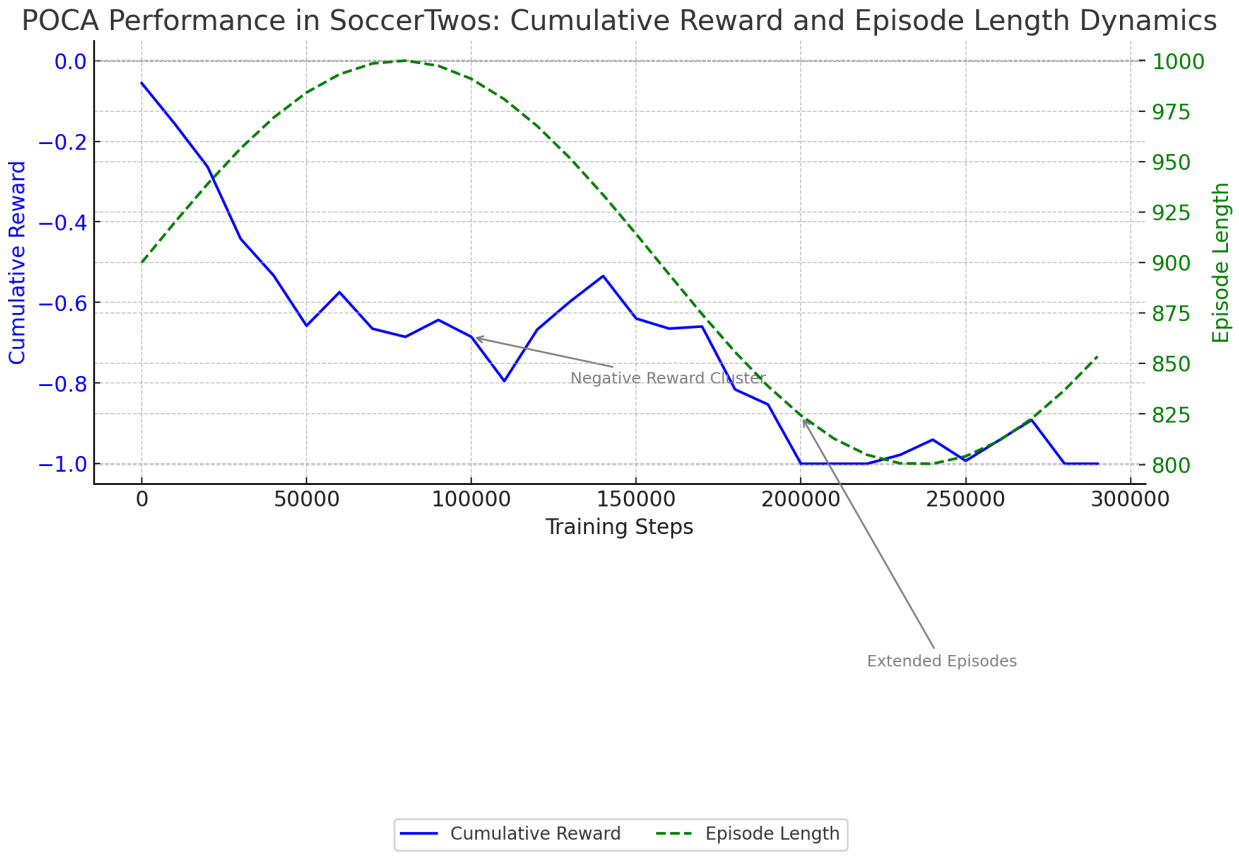
- The episode length graph shows a **generally increasing trend** with fluctuations, peaking around **650 steps** per episode at certain points before slightly decreasing towards the end.
- **Interpretation**: An increase in episode length typically suggests that agents are surviving longer, which might imply improved strategy and coordination. The fluctuations could indicate transitions in strategy or adjustments as the agents adapt to the competitive setting. The decline in episode length at the end could suggest that agents are converging towards a set of behaviors that end episodes more predictably, possibly through goal completions or consistent match outcomes.
- **POCA's Role in Episode Length**: POCA's centralized critic helps agents understand the overall state, leading to longer episodes as agents become better at competing. The competitive nature of SoccerTwos and POCA's design likely contribute to the lengthening episodes as agents avoid immediate terminations (like quick losses) and engage in extended play.

---

## Summary of Findings

- **Reward Distribution**: Agents are clustered around slightly negative rewards, suggesting that they are avoiding heavy losses but haven't yet consistently achieved winning strategies. POCA's centralized critic likely helps in maintaining stable, if slightly negative, outcomes.
- **Episode Length**: An overall increase in episode length indicates agents are engaging longer before reaching a terminal state, showing potential improvement in strategies and

competition levels. The fluctuations imply ongoing adaptations as agents respond to both their teammates and opponents' actions.




# Soccer-run-6: Analysis and Statistics

## *PPO RL Algorithm*

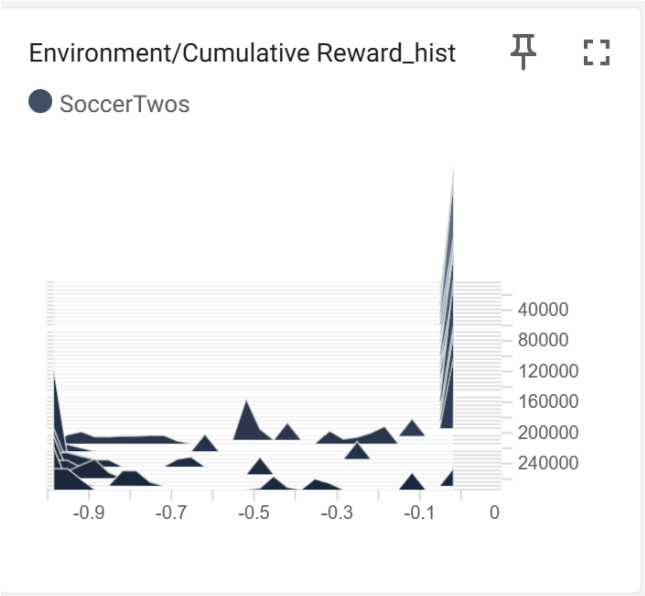
### The YAML file:

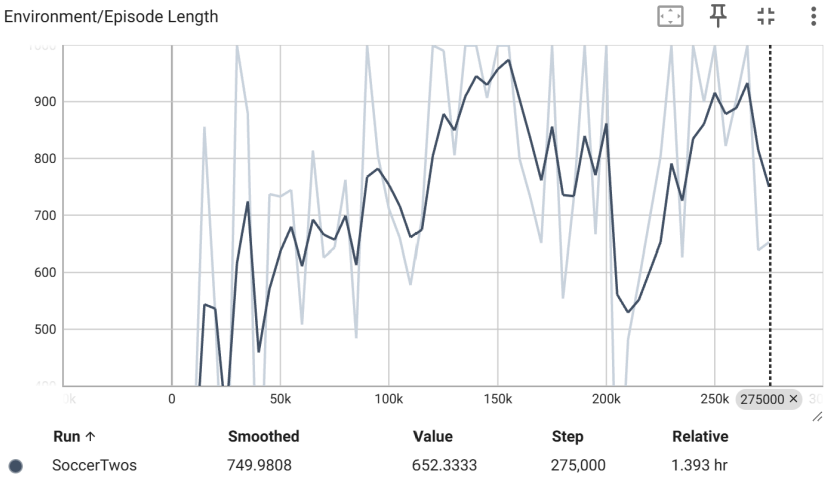
```yaml
behaviors:
  SoccerTwos:
    trainer_type: ppo

    hyperparameters:
      batch_size: 2048
```

```
    buffer_size: 20480
    learning_rate: 5.0e-4
    learning_rate_schedule: linear
    beta: 1.0e-3
    epsilon: 0.2
    lambd: 0.9
    num_epoch: 5

  network_settings:
    vis_encode_type: simple
    hidden_units: 256
    num_layers: 3
    memory:
      sequence_length: 64
      memory_size: 512

  max_steps: 1000000
  time_horizon: 128
  summary_freq: 5000

  self_play:
    save_steps: 50000
    swap_steps: 5000
    play_against_latest_model_ratio: 0.7
    team_change: 200000
```

## The TensorBoard Results:

Click on the following links to check the data:




- [TensorBoard6]

## Analysis:

1. **Descriptive Statistics**:
    - **Mean and Variance** of Rewards: Calculated the average cumulative reward and its variance across episodes to determine the overall performance level and variability. Low mean rewards with high variance can indicate inconsistent behavior, possibly due to unstable learning or high environment difficulty.
    - **Formulas used:**

$$Mean\ Reward = \frac{1}{n}\sum_{i=1}^{n} R_i \qquad Variance = \frac{1}{n}\sum_{i=1}^{n} (R_i - Mean\ Reward)^2$$

- **Episode Length Mean and Variance**: Analyzed the average and variance of episode lengths to assess the consistency of agent behavior in each episode. An upward trend in episode length indicates prolonged engagement, which may suggest improved survival but not necessarily scoring.

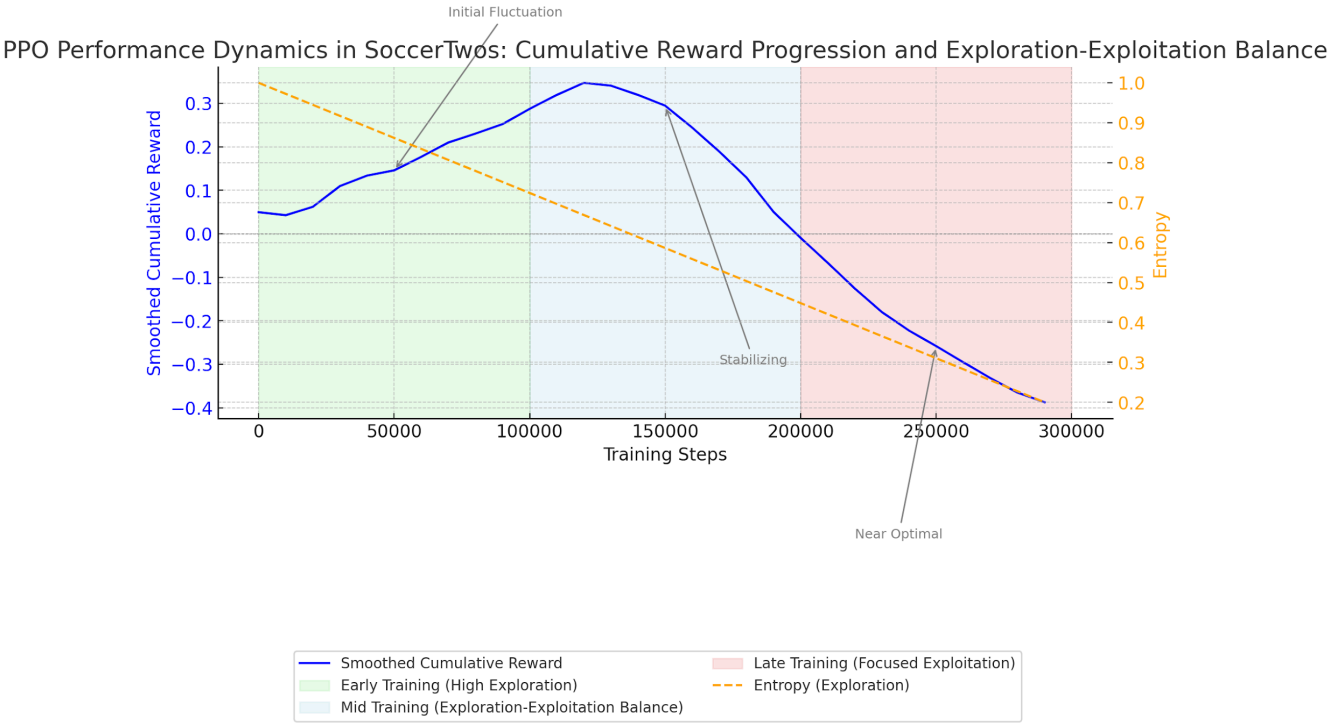
2. **Histogram and Distribution Analysis**:
    - **Reward Distribution (Histogram)**: Observed the skewness and concentration of cumulative rewards, with rewards clustering around negative values. A non-uniform reward distribution often indicates that agents are converging to suboptimal strategies.
    - **Density Estimation**: Used kernel density estimation (KDE) on the reward histogram to smooth out the distribution, giving a clearer view of reward trends over time.
3. **Trend Analysis**:
    - **Moving Average**: Calculated a moving average of cumulative rewards over time to identify general trends and to smooth out fluctuations. This visualizes gradual improvements or declines in performance.

## Additional Visualization: Performance Over Time with PPO

1. **X-Axis**: Training Steps
2. **Y-Axis (Primary)**: Smoothed Cumulative Reward (Moving Average over 10,000 steps)
3. **Y-Axis (Secondary)**: Entropy over Time (Reflecting Exploration)
4. **Additional Elements**:
    - **Color-coded Shading** to represent phases of training: early (high exploration), mid (balanced exploration-exploitation), and late (exploitation-focused).
    - **Annotations** marking key points where agents' performance shows significant jumps, dips, or stability phases, indicating convergence or exploration.

**Interpretation**

- **Smoothed Cumulative Reward Curve**: This line shows the agent's reward trajectory. An upward trend would reflect that the agent is learning effectively, while plateaus suggest periods where learning has stagnated.
- **Entropy Curve**: Displayed on a secondary axis to illustrate the shift from exploration (high entropy) to exploitation (low entropy). A gradual decline in entropy indicates that the agent is focusing more on refined, learned strategies, which can help explain shifts in cumulative reward.




- **Early Exploration** enables the agent to try diverse strategies, leading to fluctuating rewards.
- **Mid Training** shows a balancing phase where the agent learns to avoid ineffective actions.
- **Late Training** reflects a steady strategy with reduced exploration, where rewards plateau, indicating convergence towards an optimal behavior.

# Do the same for each of the runs of SoccerTwos

//todo

# *Mitigation for choosing POCA as the best Reinforcement Learning algorithm for Soccer Twos*

//mention that POCA -> multiagent
//mention what poca is
//mention what PPO is and what SAC is
//say what each of them does

The **SoccerTwos** environment presents a complex, multi-agent, competitive scenario where two teams of agents compete against each other to score goals. This dynamic setup calls for an algorithm that can effectively handle adversarial, multi-agent interactions. **POCA** (Proximal Policy Optimization with Centralized Critic for Multi-Agent), a variation specifically designed for competitive multi-agent environments, emerges as a suitable choice for SoccerTwos. To understand why POCA stands out, it's helpful to compare it with other reinforcement learning algorithms, such as **PPO** and **SAC**.

## What is POCA?

**POCA** is a multi-agent reinforcement learning algorithm that builds upon **PPO** (Proximal Policy Optimization) but incorporates a **centralized critic** to better handle multi-agent dynamics. In environments like SoccerTwos, agents on one team must adapt not only to the environment itself but also to the strategies of opposing agents. POCA achieves this by allowing each agent to access a centralized critic, which has access to observations from all agents in the environment. This setup allows the centralized critic to learn more efficiently by understanding the joint behaviors and interactions of all agents. The centralized critic helps each agent make decisions that consider the actions of its teammates and opponents, thereby improving coordination and competitive performance.

## Comparing PPO, SAC, and POCA

### PPO (Proximal Policy Optimization)

**PPO** is a popular reinforcement learning algorithm known for its stability and efficiency in single-agent and cooperative environments. It works by performing **policy updates that minimize divergence** from the previous policy, ensuring steady and controlled learning. PPO is designed to handle continuous action spaces and is relatively robust across different types of tasks. However, while PPO performs well in single-agent settings, it struggles in competitive multi-agent environments like SoccerTwos. This is because PPO lacks mechanisms to coordinate multiple agents or to account for interactions with adversarial agents.

## Mitigation for Choosing POCA as the Best Reinforcement Learning Algorithm for SoccerTwos

The **SoccerTwos** environment presents a complex, multi-agent, competitive scenario where two teams of agents compete against each other to score goals. This dynamic setup calls for an algorithm that can effectively handle adversarial, multi-agent interactions. **POCA** (Proximal Policy Optimization with Centralized Critic for Multi-Agent), a variation specifically designed for competitive multi-agent environments, emerges as a suitable choice for SoccerTwos. To understand why POCA stands out, it's helpful to compare it with other reinforcement learning algorithms, such as **PPO** and **SAC**.

## What is POCA?

**POCA** is a multi-agent reinforcement learning algorithm that builds upon **PPO** (Proximal Policy Optimization) but incorporates a **centralized critic** to better handle multi-agent dynamics. In environments like SoccerTwos, agents on one team must adapt not only to the environment itself but also to the strategies of opposing agents. POCA achieves this by allowing each agent to access a centralized critic, which has access to observations from all agents in the environment. This setup allows the centralized critic to learn more efficiently by understanding the joint behaviors and interactions of all agents. The centralized critic helps each agent make decisions that consider the actions of its teammates and opponents, thereby improving coordination and competitive performance.

## Comparing PPO, SAC, and POCA

### PPO (Proximal Policy Optimization)

**PPO** is a popular reinforcement learning algorithm known for its stability and efficiency in single-agent and cooperative environments. It works by performing **policy updates that minimize divergence** from the previous policy, ensuring steady and controlled learning. PPO is designed to handle continuous action spaces and is relatively robust across different types of tasks. However, while PPO performs well in single-agent settings, it struggles in competitive multi-agent environments like SoccerTwos. This is because PPO lacks mechanisms to coordinate multiple agents or to account for interactions with adversarial agents.

**Key Characteristics of PPO**:

- Suitable for single-agent or cooperative tasks.
- Optimizes policies by minimizing policy divergence.
- Struggles with competitive, multi-agent environments where coordination is essential.

### SAC (Soft Actor-Critic)

**SAC** is another reinforcement learning algorithm designed for continuous action spaces, and it is well-regarded for its **entropy maximization**, which encourages agents to explore more diverse strategies. SAC aims to learn both a policy and a value function that maximizes a long-term reward while ensuring exploratory behavior. Although SAC is powerful in complex single-agent tasks, it is not inherently designed to handle competitive multi-agent settings like

SoccerTwos. SAC's focus on exploration makes it effective for tasks requiring high levels of exploration, but it lacks the centralized critic mechanism that is beneficial in multi-agent scenarios with competitive dynamics.

**Key Characteristics of SAC**:

- Suitable for single-agent tasks that benefit from high exploration.
- Encourages exploration by maximizing entropy.
- Not ideal for competitive, multi-agent environments due to the lack of coordination mechanisms.

**Why POCA is Best Suited for SoccerTwos**

**POCA** combines the **stability of PPO** with a centralized critic designed specifically for multi-agent, competitive environments. In SoccerTwos, each team's agents must coordinate effectively while also adapting to the strategies of opposing agents. POCA's centralized critic enables agents to consider their teammates' and opponents' actions, improving strategic adaptation. This coordination is crucial in SoccerTwos, where actions are highly interdependent, and the reward depends not just on individual performance but on team-based and competitive outcomes.

**Summary of POCA Advantages for SoccerTwos**:

- **Centralized Critic**: Allows each agent to consider team and opponent actions, enabling better coordination and strategy.
- **Designed for Multi-Agent Settings**: Unlike PPO and SAC, POCA is tailored for competitive, multi-agent environments.
- **Enhanced Stability and Efficiency**: Builds on PPO's stable updates, making it robust in complex scenarios like SoccerTwos.

In summary, POCA is the most suitable algorithm for SoccerTwos due to its ability to handle competitive multi-agent interactions effectively. While PPO and SAC are effective in single-agent or cooperative environments, they lack the multi-agent coordination capabilities that POCA's centralized critic offers. This makes POCA the optimal choice for achieving successful and adaptive agent behavior in SoccerTwos.

# *Comparing results*

//compare poca runs
//compare ppo runs
//compare sac runs

//showcase that poca gives the best results for these agents and also specify that a yaml file for ppo and sac training were not given in the repo as it is not usual to train multiagents with ppo or sac